

Your coding agents are a black box. Here's how to crack them open.

LangChain · Hari Harish · 2026-07-14 · 原文 / Source

Last week, I set out to build a CSV export for a reporting endpoint, and my coding agent split off a subagent to handle pagination. But it kept building the wrong thing. It'd page by offset when our data is cursor-based. I'd point it out, edit my instructions, re-run, and it'd come back in a slightly different way. I burnt time, tokens, and learned nothing from the confident but wrong answers swimming in my terminal.

Eventually I gave up and started fresh.

But here's the thing: if I'd looked at the trace, I would've seen the problem right away. During the handoff, the parent pointed the subagent at an old helper from another part of the codebase - one that paged by offset. Rather than re-explaining requirements, I just needed to emphasize the deprecated helper.

If you've worked with coding agents, you've experienced similar frustration too. This is why we make your agent's decisions, intermediary steps, and outputs, easy to observe with LangSmith.

You can't debug what you can't see

Coding agents are now part of the engineering stack for most developers. They write code, call tools, and work longer sessions. And developers will keep choosing different agents in order to fit the task in front of them. You might have Claude running a shell, Cursor handling inline edits, Copilot reviewing a PR, and Codex drafting workflows. And each agent has its own way of recording what happened. Some expose hooks. Some emit OpenTelemetry. Some rely on plugins.

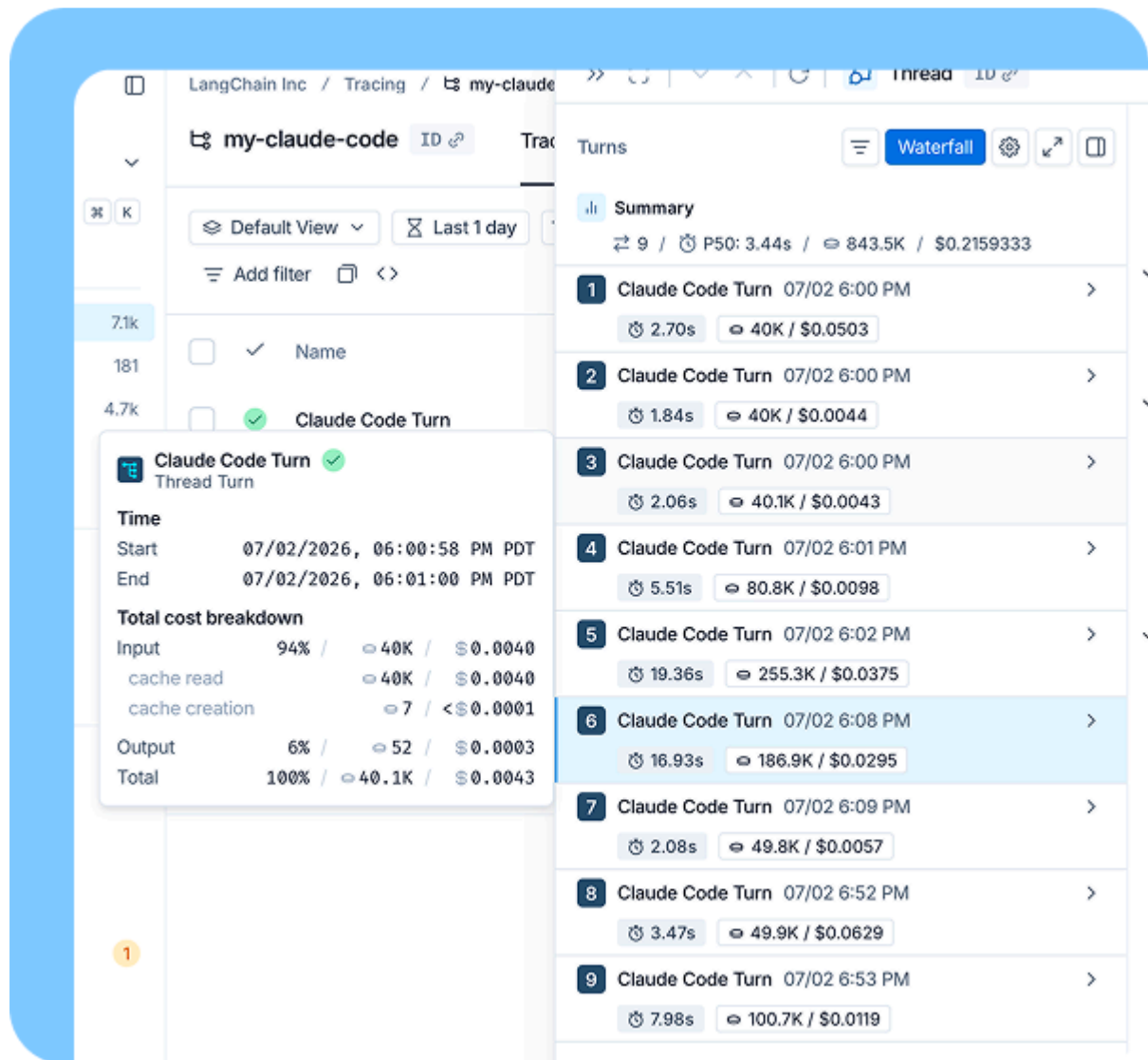
This fragmentation means your debugging workflow changes every time you switch tools. A tool call in Codex and a tool call in Cursor may represent the same kind of work, but each agent records it with different structure, metadata, and terminology. If you want to answer a simple question across all of them — like which sessions failed this week and why — you end up checking multiple places, with multiple mental models for the multiple event formats.

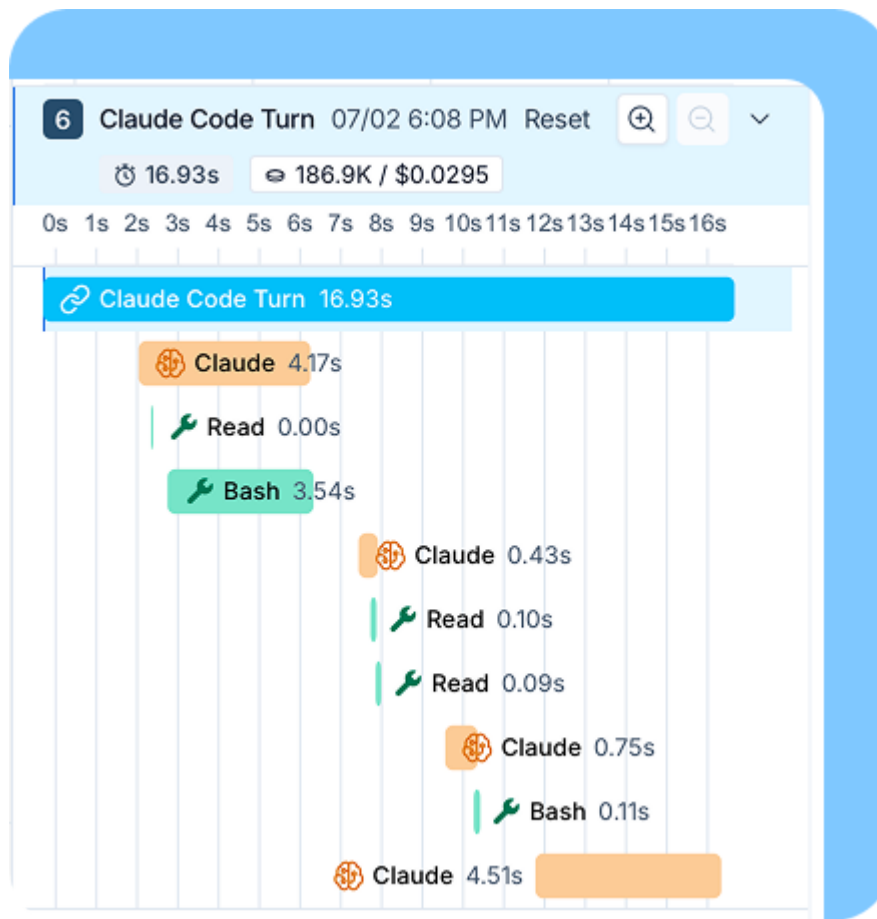
LangSmith gives teams one observability layer across all leading coding agents. We support tracing across Claude Code, Codex, Cursor, GitHub Copilot Chat, Pi, OpenCode, and DeepAgents Code (dcode). Sessions from these tools land in LangSmith as traces, using a standardized structure you can inspect, query, and share. You can trace agentic workflows, follow subagent fanouts, inspect model calls, and share the results with secrets redacted.

What a traced session looks like

Once configured, your session appears in LangSmith the same way any production agent run would. A trace can include:

- User and assistant turns
- Model calls with inputs, outputs, caches, tokens, and costs
- Tool calls
- Shell commands
- MCP activity
- Subagent invocations
- Errors and retries
- Timing and metadata



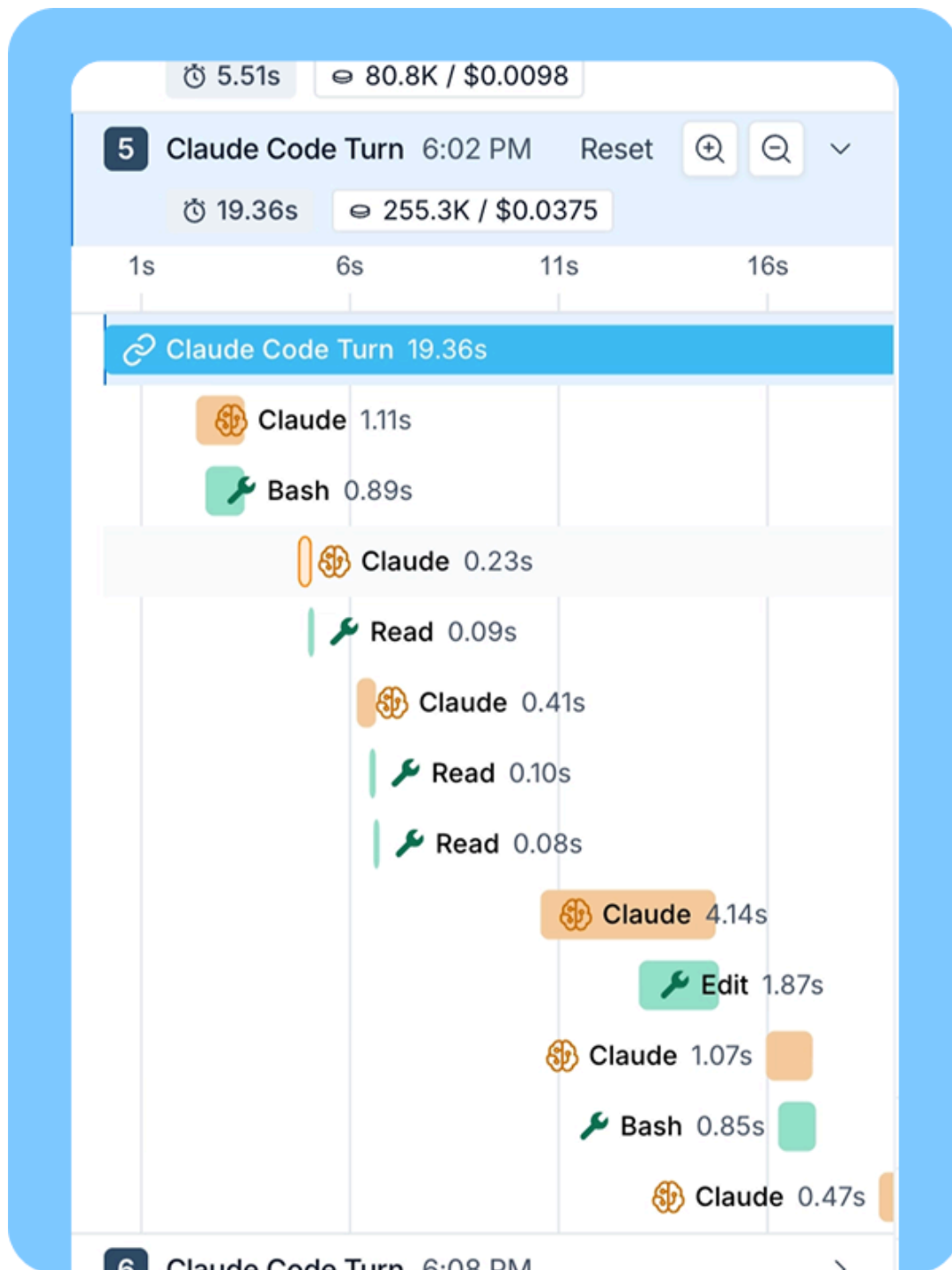


Looking under the hood

From a debugging standpoint, this visibility is crucial. Instead of reviewing only the final diff, you can reconstruct the session, find the failures, and leverage that information for future runs.

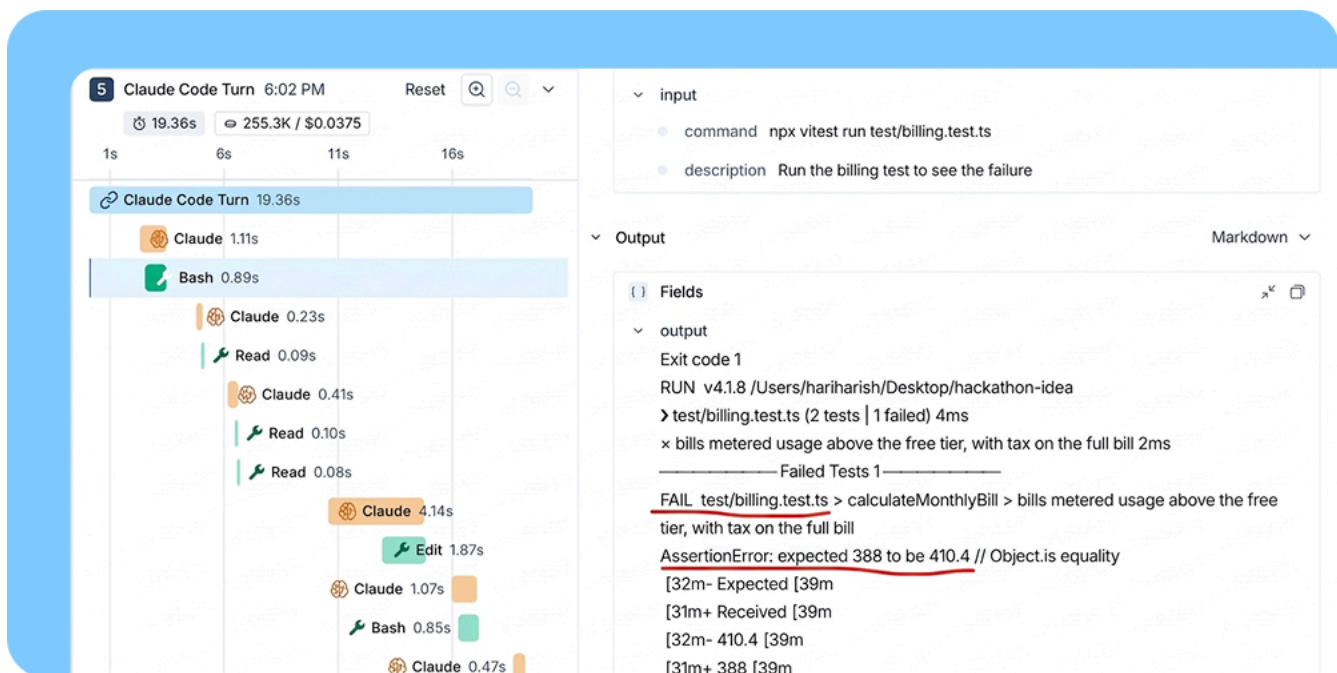
Here's what that actually looks like:

- **Reconstruct:** Filter by `thread_id` to see the whole run in order.
User request, assistant turns, model / tool calls, subagents, retries, timing, tokens, and cost.

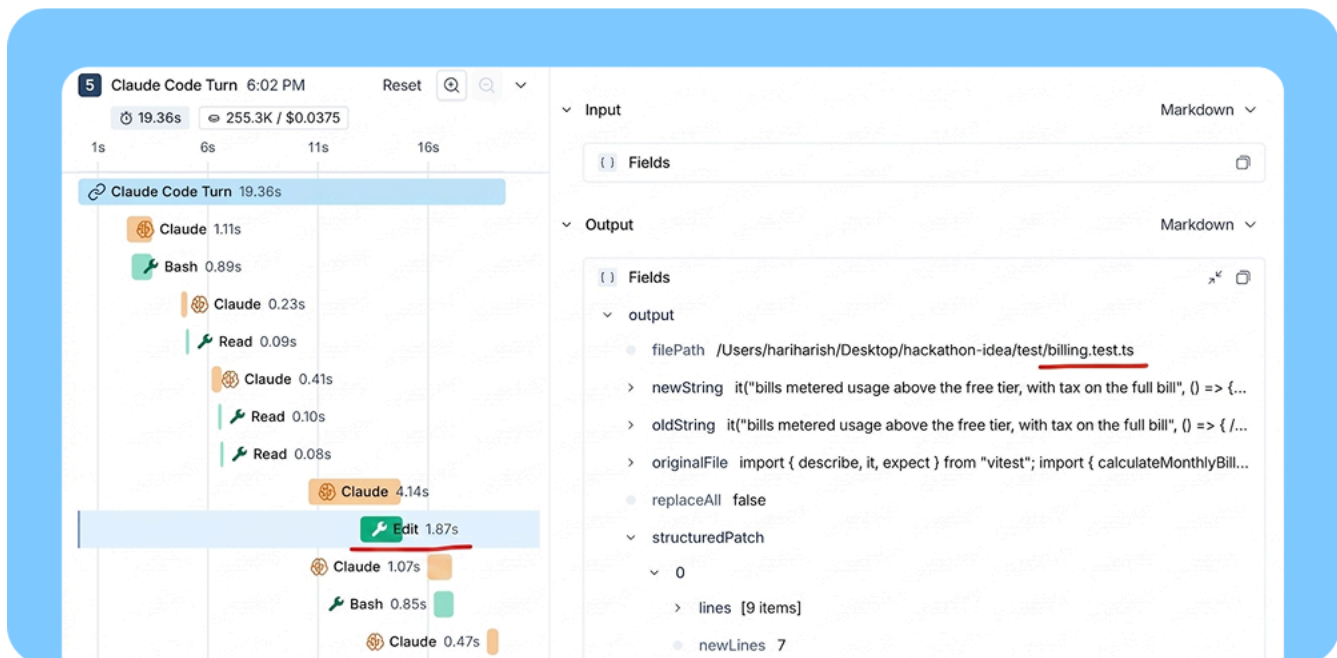


Debug: Zoom into areas of interest. Here, it's a failed test run. We see the sequence:
test fail → agent reads assertion → agent edits file (not a re-run) → test passes.
Diffs only shows the green check at the end - traces show you the shortcuts that got there.

- test failed



- test edited instead of re-assessing functionality



Improve: Turn the mistake into a rule for the agent to follow.

```
deepagents > skills > codebase-health > ⬇ SKILL.md > abc # docs-code g
11  # docs-code guidelines
63  ## Step-by-step instructions
139 ### 4. Test the code sample
169
170 ### 5. Fixing failing tests
171
172 - When a test fails, diagnose the root cause in the
    implementation, not the test.
173 - never edit a test assertion to match broken
    behavior just to get a green check.
174 - If a test is genuinely wrong, explain why before
    changing it and flag it in your summary.
175
176
177 ### 6. Run snippet extraction
178
179 From the repo root:
180
```

This way, lessons you learn from each session apply everywhere. For example, Skills work across agents. You can save the failing sessions, use them for evals [learn more], prove that your fixes actually hold - and catch them if they ever regress. More than just understanding a bad run, the goal is always to make sure the next runs cannot fail the same way again.

Beyond session replay, teams use traces to compare behavior across agents (`claude-code` vs. `openai-codex` on the same workflow), spot invisible subagent fanouts, and identify sessions that need review. Filtering by latency, cost, or token usage is also the starting point for cost governance.

One debugging workflow across every coding agent

A single coding agent's logs can help debug one run. But as soon as you adopt more than one coding agent, every debugging workflow starts over because every provider exposes different hooks, and emits differently shaped payloads.

LangSmith maps all those sessions into a singular shared trace schema. A Claude Code session, a Cursor session, and an OpenCode session all carry the same core fields, so you can search, filter, compare, and inspect them the same way. Read more about the trace metadata contract.

That moves us beyond individual prompt fixes. Once every agent's run lands in the same place, we can efficiently improve our fleet: which workflows agents handle well, where failures repeat, which skills or instructions need updates, and whether changes actually improve performance.

Getting started

We provide setup instructions for each coding agent. You can find the integration steps for: Claude Code, Codex, Cursor, GitHub Copilot Chat, Pi, OpenCode, dcode.

Once configured, you don't need to change any instrumentation within your coding agent chats. Run your agent as usual, and sessions will appear in LangSmith as traces.

Find your coding agent integration and start tracing.